

Rust Training Glossary

Rust Training — Companion Glossary

A comprehensive reference for every acronym, concept, term, library, organization, project, and proper noun that appears in this training repository. Entries are grouped by category for readability, then sorted alphabetically within each section.

Table of Contents

1. [Acronyms & Abbreviations](#)
 2. [Rust Language Concepts](#)
 3. [Rust Tools & Ecosystem](#)
 4. [Crates & Libraries](#)
 5. [Projects in This Repository](#)
 6. [Web & Networking Concepts](#)
 7. [Software Engineering Concepts](#)
 8. [Organizations & Companies](#)
 9. [Computing & Systems Concepts](#)
 10. [Game & Domain Concepts](#)
-

Acronyms & Abbreviations

Acronym	Stands For	Brief Definition
API	Application Programming Interface	A defined contract through which software components communicate. In Rust web projects, typically a set of HTTP endpoints.
Arc	Atomically Reference Counted	A thread-safe smart pointer type (<code>std::sync::Arc</code>) that enables multiple owners of the same data across threads. See also: <i>Reference Counting</i> , <i>Mutex</i> , <i>RwLock</i> .
ASCII	American Standard Code for Information Interchange	A 7-bit character encoding standard that assigns numeric values to letters, digits, and symbols. Used in the Blackjack card-flip animation and TUI displays.
AWS	Amazon Web Services	Amazon's cloud-computing platform. Rust is used in parts of its infrastructure.
CE	Close Encounter	A classification system for UFO/UAP sightings (CE1 = visual sighting at close range, CE2 = physical trace, CE3 = entity sighted, CE4 = abduction, CE5 = human-initiated contact). Used in the Anomalous Inquiry project.
CLI	Command-Line Interface	A text-based interface where the user types commands. Many projects in this repo offer a <code>--cli</code> flag to run without a GUI.

Acronym	Stands For	Brief Definition
CPU	Central Processing Unit	The primary processor in a computer. Rust compiles to native machine code that runs directly on the CPU with no interpreter or virtual machine between.
CSS	Cascading Style Sheets	A language for describing the visual presentation of HTML documents. Tailwind CSS is used in the Esoteric Wisdom project.
GATs	Generic Associated Types	An advanced Rust type-system feature (stabilised in Rust 1.65) that allows associated types in traits to be generic over lifetimes or other parameters.
gRPC	Google Remote Procedure Call	A high-performance RPC framework that uses Protocol Buffers for serialisation. Used via the tonic crate in Rust.
GUI	Graphical User Interface	A visual interface using windows, buttons, and graphics instead of text commands. Built with eframe/egui throughout this repository.
HTML	HyperText Markup Language	The standard language for structuring web pages. Rendered server-side in the Axum/Tera and Axum/Askama web projects.
HTTP	HyperText Transfer Protocol	The application-layer protocol used to transfer data on the web. Requests and responses flow between clients (browsers) and the Axum server.
HTTPS	HTTP Secure	HTTP layered over TLS encryption. Required for production deployments to protect data in transit.
I/O	Input / Output	Reading from or writing to any external resource — disk, network, terminal. Rust's <code>async / await</code> is optimised for high-concurrency I/O.
IDE	Integrated Development Environment	A full-featured code editor with build, debug, and refactoring tools (e.g., VS Code with <code>rust-analyzer</code>).
JSON	JavaScript Object Notation	A lightweight, human-readable data format widely used for configuration and data exchange. Used extensively with <code>serde_json</code> to drive Iron Age RPG content.
JWT	JSON Web Token	A compact, self-contained token format for securely transmitting claims between parties. Used in Esoteric Wisdom for stateless authentication.
LSP	Language Server Protocol	A protocol that allows editors to communicate with a language intelligence server. <code>rust-analyzer</code> is the Rust LSP implementation.
MVC	Model-View-Controller	An architectural pattern separating data (Model), presentation (View), and request handling (Controller). The Axum web projects loosely follow this separation.
NPC	Non-Player Character	An in-game character controlled by the computer, not the human player. Defined in the character crate of Iron Age RPG.
OS	Operating System	Software that manages hardware and provides services to programs (e.g., Linux, Windows,

Acronym	Stands For	Brief Definition
		macOS). Rust's std library abstracts over OS differences.
PDF	Portable Document Format	A file format for fixed-layout documents. <code>printpdf</code> is used in Anomalous Inquiry to export articles.
PHC	Password Hashing Competition	An open competition (2013–2015) that selected Argon2 as the recommended password hashing algorithm.
PR	Pull Request	A mechanism on GitHub for proposing code changes and requesting review before merging into a main branch.
RAM	Random Access Memory	Volatile, fast memory where running programs store their data. Rust's ownership model gives precise control over RAM allocation and deallocation.
REPL	Read-Eval-Print Loop	An interactive environment that reads an expression, evaluates it, prints the result, and loops. Rust has <code>evcxr</code> for a REPL experience.
RPC	Remote Procedure Call	A protocol that allows a program to execute a procedure on a remote server as if it were local.
RPG	Role-Playing Game	A game genre in which players assume the roles of characters in a fictional world, progressing through quests and combat. The Iron Age RPG is this repository's flagship project.
RSS	Really Simple Syndication	An XML-based web feed format for publishing frequently updated content. Anomalous Inquiry generates an RSS feed automatically.
RwLock	Read-Write Lock	A synchronisation primitive that allows multiple concurrent readers <i>or</i> one exclusive writer. Used as <code>Arc<RwLock<AppState>></code> in the Esoteric Wisdom web server.
SDK	Software Development Kit	A collection of tools, libraries, and documentation for building on a particular platform.
SQL	Structured Query Language	The standard language for relational database queries. <code>sqlx</code> and <code>SeaORM</code> are Rust crates for async SQL access.
SSH	Secure Shell	A cryptographic network protocol for operating network services securely over an unsecured network.
SSR	Server-Side Rendering	Generating HTML on the server and sending it to the client, as opposed to rendering in the browser with JavaScript. Both web projects in this repo use SSR exclusively.
STD	Standard Library	Rust's built-in library (<code>std</code>), providing core types, I/O, threading, and more. Contrast with <code>no_std</code> for embedded targets.
TCP	Transmission Control Protocol	A reliable, ordered, connection-oriented transport protocol used by HTTP.

Acronym	Stands For	Brief Definition
TLS	Transport Layer Security	A cryptographic protocol that provides privacy and data integrity over a network (successor to SSL).
TOML	Tom's Obvious, Minimal Language	A simple configuration file format used by Cargo (<code>Cargo.toml</code>) and for Iron Age RPG data assets.
TUI	Terminal User Interface	A text-based but visually structured interface in a terminal, using cursor positioning and colour (e.g., the Minesweeper board built with <code>crossterm</code>).
UAP	Unidentified Aerial Phenomenon	The official US government term for what was previously called UFO. Used in the Anomalous Inquiry research platform.
UDP	User Datagram Protocol	A connectionless, low-latency transport protocol. Less common in web servers, but relevant in game networking.
UFO	Unidentified Flying Object	The colloquial term for unidentified aerial phenomena. See <i>UAP</i> .
URL	Uniform Resource Locator	A web address identifying a resource (e.g., <code>https://esoteric-wisdom.onrender.com</code>).
UUID	Universally Unique Identifier	A 128-bit identifier standard used to uniquely identify resources without a central registry.
VM	Virtual Machine	An emulated computer system. Unlike Java or Python, Rust compiles to native code — there is no Rust VM.
WASM / Wasm	WebAssembly	A binary instruction format that runs in browsers at near-native speed. Rust can compile to Wasm as a compilation target.
WIP	Work In Progress	A project or branch that is actively being developed and not yet ready for final review.
XML	Extensible Markup Language	A general-purpose markup language. RSS feeds are XML documents.
YAML	YAML Ain't Markup Language	A human-readable data serialisation format commonly used in configuration files (e.g., <code>render.yaml</code>).

Rust Language Concepts

Async / Await

Keywords (`async fn`, `.await`) that allow writing asynchronous code in a synchronous style. An `async fn` returns a `Future`; `.await` pauses execution until the `Future` resolves. Powered at runtime by an executor such as *Tokio*. Used in all three web projects to handle concurrent HTTP requests efficiently.

Borrow Checker

The part of the Rust compiler that enforces the ownership and borrowing rules at compile time. It tracks when and where references are used, preventing data races, dangling pointers, and use-after-free errors before the program ever runs.

Borrowing

Temporarily granting access to a value without transferring ownership. A shared reference (`&T`) allows reading; a mutable reference (`&mut T`) allows modification. The borrow checker enforces that you can have *many* shared references *or* exactly *one* mutable reference — never both simultaneously.

Closure

An anonymous function that captures variables from its enclosing scope. Written as `|args| expression`. Common in iterator chains: `vec.iter().map(|x| x * 2).collect()`.

Crate

The fundamental compilation unit in Rust — equivalent to a library or executable package. Every `Cargo.toml` describes one crate. “Upstream crate” means a dependency; “downstream crate” means a crate that depends on yours.

Dangling Pointer

A pointer that refers to memory that has already been freed. Rust’s ownership system makes it impossible to create dangling pointers in safe code — the compiler rejects them at compile time.

Data Race

A concurrent bug where two threads access shared data simultaneously, and at least one access is a write, without synchronisation. Rust’s ownership rules and type system (Send / Sync traits) make data races in safe code a compile-time error.

Derive Macro

An attribute (`#[derive(...)]`) that automatically implements a trait for a struct or enum. Common examples: `#[derive(Debug, Clone, Serialize, Deserialize)]`. Saves boilerplate and reduces the chance of errors.

Drop

The mechanism by which Rust automatically frees resources (memory, file handles, network sockets) when a value goes out of scope. The Drop trait lets you customise this behaviour. It is deterministic — unlike garbage collection, you know exactly *when* drop runs.

Edition

A mechanism for introducing backward-incompatible language changes without breaking existing code. Each crate declares its edition in `Cargo.toml` (e.g., `edition = "2021"`, `edition = "2024"`). Editions coexist in the same project; a dependency can use edition 2021 while your crate uses 2024.

Enum (Enumeration)

A type that can be one of several named variants. Rust enums are far more powerful than C enums — each variant can carry data of different types. Used for game states, error types (`GameError`), damage types (`DamageType`), and status effects (`StatusEffect`).

Expression vs Statement

In Rust, most constructs are *expressions* (they evaluate to a value). A *statement* performs an action but does not return a value. The last expression in a block is its return value (no `return` keyword needed). This distinction matters in function bodies and `match` arms.

Feature Flags

Optional capabilities in a crate declared in `Cargo.toml`. A downstream crate opts in with `features = ["feature_name"]`. Example: `eframe = { features = ["default_fonts", "glow"] }`. Reduces compile time and binary size by excluding unneeded code.

Future

A value that represents an asynchronous computation that hasn't completed yet. `async fn` functions return `impl Future<Output = T>`. Futures are lazy — they do nothing until `.await`-ed or polled by an executor.

Garbage Collector (GC)

An automatic memory management system used by languages like Java, Python, and Go. It periodically finds and frees unreachable memory, but introduces runtime pauses. Rust has **no garbage collector** — memory is managed through ownership and `drop`.

Generic

A function, struct, enum, or trait parameterised over one or more types. Written with angle brackets: `fn largest<T: PartialOrd>(list: &[T]) -> &T`. Generics are *monomorphised* at compile time — the compiler creates a specialised copy for each concrete type used, producing zero-overhead abstractions.

HRTB (Higher-Ranked Trait Bounds)

A type-system feature for expressing constraints over *all possible lifetimes*. Written as `for<'a> Fn(&'a str)`. Advanced usage, listed as a “next steps” goal in the presentation.

impl Trait

Short for “implement trait”. Used in two ways: (1) as a function parameter type (`fn f(x: impl Display)`) meaning “any type that implements `Display`”; (2) as a return type (`fn f() -> impl Iterator`) meaning “some concrete type that implements `Iterator`” without naming it.

Iterator

A value that produces a sequence of items one at a time. The `Iterator` trait provides dozens of adapters: `map`, `filter`, `take`, `zip`, `enumerate`, `flat_map`, `fold`, `sum`, `collect`, and many more. Iterator chains are *lazy* — items are produced only as consumed — and *zero-cost* — they compile to the same code as a manual loop.

Lifetime

A label on a reference that tells the compiler how long the reference is valid. Written with a tick mark: `'a`. The borrow checker uses lifetimes to ensure references never outlive the data they point to. Most lifetimes are inferred; explicit annotations are required when the compiler can't figure it out.

Macro

Code that generates code at compile time. Rust has two kinds: *declarative macros* (`macro_rules!`) and *procedural macros* (attribute macros like `#[derive(...)]` and function-like macros). `println!`, `vec!`, `match!`, `format!` are declarative macros. `#[derive(Deserialize)]` is a procedural macro.

Match

A control-flow construct that pattern-matches against an expression and executes the arm whose pattern fits. It is exhaustive — the compiler forces you to handle every possible variant of an enum. More powerful than `switch` in C or Java.

Monomorphisation

The compiler process of replacing generic type parameters with concrete types. A function `fn f<T>(x: T)` called with `i32` and `String` produces two compiled copies. This is how Rust achieves zero-cost abstractions from generics.

Move Semantics

When a value is assigned to a new variable or passed to a function, ownership *moves* — the original binding is invalidated. This prevents double-free bugs. Types that implement `Copy` (small, stack-allocated types like `i32`, `bool`, `char`) are copied instead of moved.

Mutex

Mutual exclusion lock (`std::sync::Mutex<T>`). Wraps a value so only one thread can access it at a time. Typically used as `Arc<Mutex<T>>` to share mutable state across threads. The `lock()` method returns a guard that auto-releases when dropped.

no_std

A mode for writing Rust without the standard library, for use on embedded systems or kernels where an OS and allocator are unavailable. Only the core library (which has no heap allocation) is available. Listed as a future goal in the presentation.

Ownership

Rust's central memory management concept. Every value has exactly one *owner* (a variable or struct field). When the owner goes out of scope, the value is automatically dropped (freed). Ownership can be *moved* to a new owner or temporarily *borrowed* via references.

Panic

An unrecoverable error that terminates the current thread (or the whole program in a single-threaded context). Triggered by `panic!("message")`, out-of-bounds array access, or integer overflow in debug mode. Should be avoided in library code — use `Result<T, E>` instead.

Pattern Matching

Deconstructing and testing values against a pattern. Used in `match`, `if let`, `while let`, and `let` bindings. Can match literals, tuple structs, enum variants, ranges, and combine patterns with `|` and guards.

Procedural Macro

A Rust macro that runs as a Rust program at compile time to transform syntax. Includes `#[derive(...)]` macros (like those in `serde`), attribute macros (like those in `tokio` and `axum`), and function-like macros.

Result<T, E>

An enum with two variants: `Ok(T)` (success, containing value of type `T`) and `Err(E)` (failure, containing error of type `E`). The idiomatic Rust way to handle fallible operations. There are no exceptions — every error must be explicitly handled or propagated. The `?` operator is syntactic sugar for “return `Err` if this is `Err`, otherwise unwrap the `Ok` value.”

Send / Sync

Marker traits that control thread safety. `Send` means a type can be transferred to another thread. `Sync` means a type can be shared (via `&T`) between threads. The compiler automatically implements these for types that are safe, and refuses to compile code that would violate thread safety.

Shadowing

Declaring a new variable with the same name as an existing one, which “shadows” the original within the new scope. Unlike mutation, shadowing can change the type of the binding. Example: `let x = "5"; let x: i32 = x.parse().unwrap();`

Slice

A view into a contiguous sequence of elements in memory, written as `&[T]` or `&str`. A slice does not own the data it refers to. `String` vs `&str`, `Vec<T>` vs `&[T]` — slices are the borrowed, view form of the owned types.

Smart Pointer

A data structure that acts like a pointer but provides additional capabilities such as ownership (`Box<T>`), reference counting (`Rc<T>`, `Arc<T>`), interior mutability (`Cell<T>`, `RefCell<T>`), or locking (`Mutex<T>`, `RwLock<T>`).

Stack vs Heap

The **stack** is fast, fixed-size memory for local variables. The **heap** is dynamic memory for data whose size is unknown at compile time. `Box<T>` allocates on the heap. Rust’s ownership model applies equally to both; heap memory is freed when its owning `Box/Vec/String` is dropped.

String vs &str

`String` is an owned, growable, heap-allocated UTF-8 string. `&str` is a borrowed reference to a string slice (often pointing into a `String` or a string literal baked into the binary). Function parameters that don’t need ownership should accept `&str`; functions that need to own or grow the string use `String`.

Struct

A named data type with named fields. Used to model domain entities throughout the codebase: `Stats`, `Character`, `Enemy`, `Quest`, `Item`. Methods are added via `impl` blocks.

Trait

Rust's mechanism for defining shared behaviour — similar to interfaces in Java or Go, or type classes in Haskell. A trait declares method signatures; types implement the trait by providing concrete implementations. Traits enable polymorphism without inheritance.

Trait Object

A dynamically dispatched trait reference: `Box<dyn TraitName>` or `&dyn TraitName`. The concrete type is erased at compile time; dispatch happens at runtime via a vtable. Used in the Ultra Game Suite so the GUI launcher can hold any game variant through `Box<dyn Game>`.

Type Inference

The compiler's ability to deduce the type of a variable without an explicit annotation. `let x = vec![1, 2, 3];` — the compiler infers `Vec<i32>`. Type inference is local: the compiler uses surrounding context, not whole-program analysis.

Unit Type ()

The type with exactly one value, also written `()`. Functions that don't return a meaningful value return `()`. Equivalent to `void` in C, but it's an actual type in Rust's type system.

Unsafe

A keyword (`unsafe { ... }`) that unlocks operations the compiler cannot verify: dereferencing raw pointers, calling C functions via FFI, implementing unsafe traits. All production Rust code should minimise unsafe blocks and clearly document why they are sound.

Use-After-Free

A memory bug where a program accesses memory after it has been freed. Rust's ownership system makes this a compile-time error in safe code.

Vec

A growable, heap-allocated array. The most commonly used collection in Rust. Analogous to `ArrayList` in Java or `std::vector` in C++. Elements are accessed by index; the `Vec` owns its elements and frees them when dropped.

Workspace

A Cargo feature for managing multiple crates together in one repository. A root `Cargo.toml` with `[workspace]` lists member crates. Shared dependencies are declared once under `[workspace.dependencies]`. Iron Age RPG uses a workspace of ten crates.

? Operator

Shorthand for propagating errors. In a function returning `Result<T, E>`, writing `expr?` means “if `expr` is `Err(e)`, return `Err(e.into())` immediately; otherwise unwrap the `Ok(v)` value and continue.” Makes error-handling code concise and readable without hiding errors.

Rust Tools & Ecosystem

Cargo

Rust's official build system and package manager. Handles compiling code (`cargo build`), running tests (`cargo test`), downloading dependencies (`cargo add`), generating documentation (`cargo doc`), and publishing crates (`cargo publish`). The `Cargo.toml` manifest and `Cargo.lock` lockfile live at the project root.

Cargo.lock

An auto-generated file that records the exact version of every dependency (direct and transitive) that was used in the last successful build. Should be committed for applications; often `.gitignore`d for libraries.

Cargo.toml

The manifest file for a Rust crate. Declares the crate name, version, edition, authors, dependencies, binary targets, feature flags, and workspace membership.

clippy

Rust's official linter. Catches common mistakes, suggests more idiomatic code, and enforces best practices. Run with `cargo clippy`. Much more opinionated than the base compiler warnings.

crates.io

The official public registry for Rust crates (packages). When you add a dependency to `Cargo.toml`, Cargo downloads it from crates.io by default. The Rust equivalent of npm (JavaScript) or PyPI (Python).

docs.rs

A service that automatically builds and hosts API documentation for every crate published to crates.io. Available at <https://docs.rs/<crate-name>>.

evcxr

A Rust REPL (Read-Eval-Print Loop) and Jupyter kernel, allowing interactive Rust evaluation without a full project. Useful for experimentation.

fmt (rustfmt)

Rust's official code formatter. Enforces a consistent, community-agreed style. Run with `cargo fmt`. No argument about braces or indentation — just run the formatter.

miri

An interpreter for Rust's mid-level intermediate representation (MIR) used to detect undefined behaviour in unsafe code, including memory leaks, out-of-bounds access, and use-after-free.

Nightly

An unstable release channel for Rust, updated daily, containing the very latest compiler features before they are stabilised. Stable and beta channels are also available. Features like GATs and some procedural macro capabilities landed in nightly before reaching stable.

rustc

The Rust compiler. Converts Rust source code (`.rs`) through several stages — parsing, type checking, borrow checking, MIR lowering, LLVM IR generation, and finally machine code — to produce a native binary.

rustup

The Rust toolchain installer and version manager. Use it to install Rust, switch between stable/nightly/beta channels, add compilation targets (e.g., `wasm32-unknown-unknown`), and manage components like `clippy`, `rustfmt`, and `rust-analyzer`.

rust-analyzer

The official Rust language server (LSP implementation). Provides IDE features: autocomplete, go-to-definition, inline type hints, error highlighting, and refactoring. The standard tool for VS Code, Neovim, and most other editors.

The Book

The informal name for *The Rust Programming Language*, the official free textbook at doc.rust-lang.org/book. Covers everything from hello world to concurrency. A primary resource in this training curriculum.

Crates & Libraries

Argon2

A password hashing algorithm that won the Password Hashing Competition (2015). Deliberately slow and memory-hard to resist brute-force and GPU attacks. Used in the Esoteric Wisdom project via the `argon2` crate for securely storing user passwords.

Askama

A Jinja2-style templating engine for Rust that compiles HTML templates **at build time**. Template variables are type-checked by the Rust compiler — a broken template variable reference is a compile error, not a runtime surprise. Used in the Esoteric Wisdom project for 164 compiled templates.

Axum

A web framework for Rust built on top of Tokio and hyper. Provides routing, middleware, state extraction, and handler functions with a declarative, type-safe API. Used in both the Anomalous Inquiry and Esoteric Wisdom projects.

crossterm

A cross-platform terminal manipulation library. Provides cursor movement, colour output, keyboard input reading, and raw/alternate screen modes. Used to build the TUI for the Minesweeper game and the Blackjack

card-flip animation in the Ultra Game Suite.

eframe

The application framework layer of the egui immediate-mode GUI library. Handles window creation, event loop, and platform backend (Glow/OpenGL or wgpu). Every GUI application in this repository (guess_game_gui, Ultra Game Suite launcher, Iron Age RPG GUI) is built on eframe.

egui

“Easy GUI” — an immediate-mode GUI library for Rust. The UI is described in code each frame rather than with persistent widget objects. Handles rendering via eframe. Panels, buttons, text inputs, sliders, and custom painting are all available.

hyper

A fast, low-level HTTP library for Rust. Axum is built on top of hyper. It handles raw HTTP/1 and HTTP/2 protocol parsing and connection management.

pulldown-cmark

A Markdown parser for Rust following the CommonMark specification. Used in Anomalous Inquiry to convert Markdown articles to HTML for display.

printpdf

A Rust library for generating PDF files from scratch (text, images, graphics). Used in Anomalous Inquiry to export articles as downloadable PDFs.

rand

The standard Rust random number generation library. Provides a variety of RNG algorithms and convenience methods (gen_range, shuffle, choose). Used in the guessing game, Ultra Game Suite, and Iron Age RPG for random number generation and enemy behaviour.

Rayon

A data-parallelism library for Rust. Lets you change .iter() to .par_iter() to automatically parallelise iteration across CPU cores. Mentioned as ideal for GeoPolSim’s parallel entity processing. The borrow checker guarantees no data races in parallel iterators.

rss (crate)

A Rust library for building and parsing RSS feed documents. Used in Anomalous Inquiry to auto-generate an RSS feed of published articles.

SeaORM

An async, dynamic ORM (Object-Relational Mapper) for Rust, supporting PostgreSQL, MySQL, and SQLite. Listed as a future database-integration option.

serde

“Serialisation/Deserialisation” framework for Rust. By adding `#[derive(Serialize, Deserialize)]` to a struct, you get automatic conversion to/from JSON, TOML, YAML, MessagePack, and dozens of other formats. One of the most widely used crates in the Rust ecosystem.

serde_json

The JSON backend for `serde`. Converts Rust values to/from JSON strings or `serde_json::Value`. Used extensively to load Iron Age RPG game data from JSON asset files.

sqlx

An async, compile-time-checked SQL library for Rust. Queries are verified against your actual database schema at compile time — a SQL typo is a build error. Supports PostgreSQL, MySQL, and SQLite. Listed as a future goal.

Tailwind CSS

A utility-first CSS framework where styling is applied via composable class names directly in HTML. Used for the visual design of the Esoteric Wisdom portal (star fields, aurora effects, sacred geometry).

Tera

A Jinja2-inspired, runtime-evaluated HTML templating engine for Rust. Templates are loaded and rendered at runtime, making it easy to iterate without recompiling. Used in Anomalous Inquiry.

thiserror

A derive macro (`#[derive(Error)]`) for conveniently defining custom error types that implement `std::error::Error`. Used in Iron Age RPG’s core crate to define `GameError` and other domain errors.

Tokio

The most widely used async runtime for Rust. Provides an event loop, an async I/O scheduler, a thread pool, timers, and synchronisation primitives (`Mutex`, `RwLock`, channels). All three web projects use Tokio as their async executor.

toml (crate)

A Rust library for parsing and serialising TOML documents, typically used with `serde`. Used alongside `serde_json` to load game data assets in Iron Age RPG.

tonic

A gRPC framework for Rust built on Tokio. Generates client and server stubs from `.proto` files. Mentioned as a future goal for distributed systems work.

trpl

The `trpl` crate is a teaching helper crate specifically for “The Rust Programming Language” book (second edition). It re-exports async utilities (like `trpl::run`, `trpl::sleep`) to simplify async examples in the book’s chapters. Found in the `get-dependencies` workspace.

Projects in This Repository

Anomalous Inquiry

A fully-deployed, server-side-rendered web application built with Axum, Tera, and Tokio. A documentary-style research platform covering UAP, parapsychology, near-death experiences, and remote viewing. Notable features: zero client-side JavaScript, PDF export via `printpdf`, auto-generated RSS feed, cookie-authenticated admin panel, CE1–CE5 close encounter archive. Deployed on Render.com.

Esoteric Wisdom

The most architecturally sophisticated web project in the portfolio. A spiritual portal with 140+ content pages spanning major esoteric traditions (Hermeticism, Kabbalah, Tantra, Sufism, Druidism). Features: 164 Askama templates compiled at build time, Argon2 password hashing, JWT authentication via HTTP-only cookies, a tarot card reader with 15 historic decks, a personal journal with mood tracking, and `Arc<RwLock<AppState>>` shared state. Live at `esoteric-wisdom.onrender.com`.

Foundational Exercises

The first block of programs in `get-dependencies/projects/:hello_world`, `variables`, `functions`, `branches`, `loops`, `guessing_game`, `Arrays`, `Structs`, `TheSliceType`, `Ownership_and_Functions`, `ref_borrowing.rs`. Each exercise targets one chapter from *The Rust Programming Language*.

GeoPolSim

A geopolitical simulation engine. Models nation-state dynamics, economic interactions, and geopolitical events. Chosen to showcase Rust’s deterministic execution (no GC pauses), natural `struct/enum` data modelling, Rayon-based parallelism, Serde save/load, and a trait-based plugin architecture via the `Simulatable` trait.

Guess Game GUI (`guess_game_gui`)

An enhanced version of the classic guessing game with a graphical interface (`eframe/egui`) and roast-style banter based on how close the player’s guess is. A notable step up from the terminal-based original.

Guessing Game (`guessing_game`)

The canonical first complete Rust program from *The Rust Programming Language*. Generates a random number with `rand`, reads user input, and uses `match` on `std::cmp::Ordering` to tell the player whether their guess is too high, too low, or correct.

Iron Age RPG

The flagship project. A multi-crate Cargo workspace with ten crates: `core`, `character`, `combat`, `inventory`, `world`, `narrative`, `crafting`, `data`, `game`, and `minesweeper`. Ships as two binaries: a CLI game loop (`iron-age-rpg`) and a graphical version (`iron-age-rpg-gui`). Content: 5 sub-areas with boss encounters, 12 enemy types, 36 side quests. 84 automated tests. Data is entirely driven by JSON/TOML files.

Ultra Game Suite (`ultra_guessing_game`)

Ten complete games in one binary. GUI launcher built with `eframe`; pass `--cli` for terminal mode or `--game N` to jump to a specific game. Games: 1 — Guessing Game, 2 — Hangman, 3 — Wordle, 4 — Minesweeper, 5 — Checkers, 6 — Chess, 7 — Tic-Tac-Toe, 8 — Blackjack (with ASCII card-flip animation), 9 — Poker, 10 — Crazy Eights. Key concepts: trait objects (`dyn Game`), enum state machines, `crossterm` TUI, `rand`, `eframe`.

Web & Networking Concepts

Authentication

The process of verifying a user's identity (who are you?). Implemented in Esoteric Wisdom using Argon2 password hashing on registration and JWT tokens in HTTP-only cookies for subsequent requests.

Authorization

The process of determining what an authenticated user is allowed to do (what can you do?). A separate concern from authentication — Esoteric Wisdom's admin panel requires both authentication and admin-level authorization.

Cookie

A small piece of data stored by the browser and sent with every subsequent request to the same origin. HTTP-only cookies cannot be read by JavaScript, making them safer for storing authentication tokens. Used in Esoteric Wisdom for JWT storage.

CORS (Cross-Origin Resource Sharing)

A browser security mechanism that restricts web pages from making requests to a different domain than the one that served them. Axum middleware can configure CORS headers.

DNS (Domain Name System)

The internet's address book — translates human-readable domain names (e.g., `esoteric-wisdom.onrender.com`) to IP addresses.

HTTP-Only Cookie

A cookie with the `HttpOnly` flag set, making it inaccessible to JavaScript. Used in Esoteric Wisdom so that the JWT authentication token cannot be stolen via cross-site scripting (XSS).

Infrastructure-as-Code (IaC)

Defining and managing infrastructure (servers, databases, networking) via machine-readable configuration files rather than manual setup. `render.yaml` in the web projects is an example — it describes the deployment configuration for Render.com.

Middleware

Software that sits between the HTTP server and route handlers to process every request/response. Common uses: logging, authentication checks, CORS headers, rate limiting. Axum uses tower middleware.

Route

A URL pattern associated with a handler function. In Axum: `Router::new().route("/articles/:id", get(article_handler))`. The `:id` is a path parameter extracted by Axum.

Server-Side Rendering (SSR)

Generating complete HTML pages on the server and sending them to the client. The client receives ready-to-display HTML rather than a skeleton page that requires JavaScript to populate. Both web projects in this repo use SSR exclusively — even text-based browsers work.

Session

A server-side record of a user's authenticated state. Esoteric Wisdom avoids server-side sessions by using stateless JWTs — the authentication information is embedded in the token itself, so no session store is needed and the service is horizontally scalable.

WebAssembly (Wasm)

A binary instruction format designed to run in web browsers at near-native speed. Rust is one of the best-supported languages for compiling to Wasm. Listed as a future goal — it would allow the game logic and simulation code to run directly in a browser.

Software Engineering Concepts

Abstraction

Hiding implementation details behind a well-defined interface. In Rust, traits and modules are the primary abstraction tools. The `Simulatable` trait in `GeoPolSim` abstracts over different simulation modules; callers don't need to know the concrete type.

Automated Testing

Writing code that verifies other code behaves correctly. Rust has built-in test support: `#[test]` marks a function as a test; `cargo test` discovers and runs all tests. Iron Age RPG has 84 automated tests across its ten crates.

Binary (executable)

A compiled, runnable program. Cargo builds one binary per `[[bin]]` section in `Cargo.toml`. Iron Age RPG builds two binaries: `iron-age-rpg` (CLI) and `iron-age-rpg-gui`.

Codebase

The entire collection of source code that makes up a project. This repository's codebase spans foundational exercises, two web applications, a game suite, and a multi-crate RPG engine.

Compile Time vs Runtime

Compile time is when `rustc` processes source code and produces a binary. **Runtime** is when that binary is executing. Rust's ownership and type checks happen at compile time — errors found then are free (no running program to crash). Languages with garbage collectors or dynamic typing push more checks to runtime.

Concurrency

Multiple tasks making progress over the same time period. In Rust, concurrency is achieved through threads (`std::thread`) or async tasks (Tokio). The `Send/Sync` traits and `Arc<Mutex<T>>` pattern ensure concurrent code is memory-safe.

Data-Driven Design

A design pattern where behaviour is controlled by data (JSON, TOML, database rows) rather than hard-coded logic. Iron Age RPG uses data-driven design: enemies, quests, and items are defined in JSON/TOML files — new content can be added without touching Rust source code.

Dependency

An external crate that your project uses. Listed in `[dependencies]` in `Cargo.toml`. Cargo downloads, caches, and links dependencies automatically.

Determinism

The property of producing the same output given the same input, with no random variation in timing. Rust's lack of a garbage collector makes it more deterministic than GC languages — important for simulations (GeoPolSim) and real-time systems.

Idiomatic

Code written in the natural style of a language, using its features as intended. “Idiomatic Rust” favours `match` over long `if-else` chains, `Result`/? over exceptions, iterators over explicit loops, and traits over inheritance.

Integration Test

A test that verifies multiple components working together, as opposed to a unit test that tests one function in isolation. In Rust, integration tests live in a `tests/` directory at the crate root.

Library Crate

A crate that provides functionality to be used by other crates. Does not have a `main` function and produces no standalone executable. Most of Iron Age RPG's crates (`core`, `character`, `combat`, etc.) are library crates.

Module

A namespace within a Rust crate that groups related items. Declared with `mod module_name;` or inline with `mod module_name { ... }`. The `pub` keyword controls what is visible outside the module.

Monorepo

A single repository containing multiple related projects or packages. This repository is a monorepo: foundational exercises, Ultra Game Suite, Iron Age RPG, and web projects all live here.

Open Source

Software whose source code is publicly available for anyone to inspect, modify, and distribute. This repository is open source on GitHub. The Rust compiler, standard library, and all crates used here are open source.

Plugin Architecture

A design where new functionality can be added without modifying the core system. GeoPolSim's `Simulatable` trait is an example: new simulation modules implement the trait and plug into the engine loop cleanly.

Polymorphism

The ability to write code that works with multiple types. Rust achieves polymorphism through generics (static dispatch, zero cost) and trait objects (dynamic dispatch, small runtime overhead from vtable lookup).

Refactoring

Restructuring existing code without changing its external behaviour, to improve readability, maintainability, or performance. The borrow checker makes large Rust refactors safer than in many languages — the compiler catches broken invariants.

Regression

A bug introduced by a code change that breaks previously working functionality. Automated tests protect against regressions — if a test that was passing now fails, the change introduced a regression.

Self-Contained Binary

An executable that includes all its dependencies and can be run without a separate runtime, interpreter, or virtual machine. Rust programs compile to self-contained native binaries. *Anomalous Inquiry* and *Esoteric Wisdom* each deploy as a single binary with no runtime dependencies.

Separation of Concerns

A design principle that each module or crate should have one well-defined responsibility. Iron Age RPG's ten-crate workspace enforces this: `character` only knows about characters; `combat` only knows about combat logic.

Unit Test

A test that exercises a single function or small unit of code in isolation. In Rust, unit tests are written inside the same file as the code being tested, in a `#[cfg(test)]` module.

Zero-Cost Abstraction

A language feature that provides a high-level programming model (like iterators, generics, or closures) with no runtime overhead compared to hand-writing the low-level equivalent. Rust iterators, generics (monomorphisation), and many other features are zero-cost abstractions.

Organizations & Companies

Amazon (Amazon Web Services / AWS)

A multinational technology company and the world's largest cloud provider. Amazon uses Rust in parts of its AWS infrastructure (e.g., Firecracker VMM, Lambda) and is a major backer of the Rust ecosystem.

Apple

A consumer technology company. Apple platforms (macOS, iOS) are supported Rust compilation targets. eframe targets include macOS and iOS via the Metal graphics backend.

crates.io (operated by the Rust Foundation)

The official public package registry for Rust. See *Rust Foundation*.

Ferrous Systems

A Rust consulting and training firm, notable for contributing significantly to the Rust compiler (especially the GCC backend) and producing training materials. Not directly mentioned in the repo but prominent in the community.

Google

A multinational technology company. Google created the *Comprehensive Rust* course (a primary resource in this curriculum), uses Rust in Android (AOSP), Fuchsia OS, and Chrome. Google is a member of the Rust Foundation.

JetBrains

A software development tool company known for IntelliJ-based IDEs. Developed *RustRover* (formerly a plugin, now a standalone Rust IDE). Published the Stack Overflow Developer Survey findings referenced in the presentation.

Linux Foundation

A nonprofit organisation that supports the development of the Linux kernel. The Linux kernel began accepting Rust code in version 6.1 (2022), making Rust the second language (after C) officially supported for kernel development.

Meta (Facebook)

A multinational technology company. Meta uses Rust in some infrastructure components and is involved in the Rust ecosystem (e.g., the Buck2 build system is written in Rust).

Microsoft

A multinational technology company. Microsoft uses Rust in Windows components (e.g., parts of the Windows kernel and Azure services). Microsoft is a member of the Rust Foundation and a significant contributor to the ecosystem.

Mozilla

A nonprofit organisation best known for the Firefox browser. Mozilla Research created the Rust programming language in 2006 (Graydon Hoare’s personal project) and officially sponsored its development, leading to Rust 1.0 in 2015. Mozilla transferred stewardship to the Rust Foundation in 2021.

Render.com

A cloud hosting platform offering free and paid tiers for deploying web applications, background workers, databases, and static sites. Configuration is defined in `render.yaml` (infrastructure-as-code). Both Anomalous Inquiry and Esoteric Wisdom are deployed here.

Rust Foundation

A nonprofit organisation, incorporated in 2021, that stewards the Rust programming language and ecosystem. Founding members include Mozilla, Google, Amazon, Microsoft, and Huawei. Responsible for operating crates.io, docs.rs, and the Rust trademark.

Stack Overflow

A major Q&A website for developers. Its annual Developer Survey has named Rust the “most-loved” (now “most-admired”) programming language nine consecutive years (2016–2024), based on the percentage of users who use Rust and want to continue using it.

Computing & Systems Concepts

Allocator

The component responsible for managing heap memory — handing out memory when requested (`malloc` in C, `Box::new` in Rust) and reclaiming it when freed. Rust uses `jemalloc` or the system allocator by default. In `no_std` environments, you supply your own allocator.

Cache Line

The smallest unit of memory that a CPU cache loads from RAM, typically 64 bytes. Structuring data to fit within cache lines (cache-friendly layout) dramatically improves performance. Rust’s ownership of data layout (e.g., `#repr(C)`) enables this optimisation.

Compiler

A program that translates source code in a high-level language to machine code or an intermediate form. `rustc` is the Rust compiler. It performs lexing, parsing, name resolution, type checking, borrow checking, MIR optimisation, and LLVM code generation.

Context Switch

The process by which a CPU saves the state of one thread and loads the state of another. Context switches have overhead. Rust’s async model uses cooperative multitasking within a thread pool to achieve high concurrency with fewer context switches than spawning one OS thread per connection.

Cross-Compilation

Compiling code on one architecture/OS (the host) to run on a different architecture/OS (the target). Rust supports cross-compilation well via `rustup target add <target>`. Required for embedded and Wasm development.

FFI (Foreign Function Interface)

A mechanism that allows Rust code to call functions written in C (or other languages), and vice versa. Uses `extern "C"` blocks. Anything calling across the FFI boundary is `unsafe`.

Garbage Collection (GC)

Automatic heap memory management that periodically scans for unreachable objects and frees them. Used by Java, Python, Go, and many others. Introduces non-deterministic pause times. Rust has no GC.

LLVM

Low Level Virtual Machine — a collection of compiler technologies. `rustc` uses LLVM as its backend to generate machine code for x86, ARM, WebAssembly, and many other targets. LLVM also performs powerful optimisations.

Memory Leak

Failing to free heap memory that is no longer reachable or needed. Rust's ownership and `drop` system prevents most memory leaks automatically, though reference cycles with `Rc` can still cause leaks (mitigated by `Weak<T>`).

Microcontroller

A small integrated circuit containing a CPU, memory, and I/O all on one chip. Examples: Arduino, Raspberry Pi Pico, STM32 boards. Rust's `no_std` mode targets microcontrollers. Listed as a future goal.

Null Pointer

A pointer that points to no valid memory address (address zero). Dereferencing a null pointer is undefined behaviour in C/C++ and a crash in many languages. Rust eliminates null pointers — the `Option<T>` type (`Some(value)` or `None`) is used instead, and the compiler requires handling both cases.

Operating System Kernel

The core of an OS — manages processes, memory, and hardware. The Linux kernel accepts Rust code (since v6.1). Writing kernel code requires `no_std` and often `unsafe`.

Parallelism

Multiple tasks executing at exactly the same time on multiple CPU cores. Distinct from concurrency (multiple tasks making progress, possibly interleaved on one core). Rayon enables data parallelism in Rust.

Process

A running instance of a program with its own memory space. Rust programs are OS processes. Threads share memory within a process; separate processes communicate via IPC.

Race Condition

A bug where the outcome depends on the relative timing of events in concurrent code. A *data race* (a specific type of race condition involving unsynchronised memory access) is impossible in safe Rust.

Raw Pointer

A C-style pointer (`*const T` or `*mut T`) with no borrow-checker safety guarantees. Only usable inside `unsafe` blocks. Necessary for FFI and some low-level data structures.

Signal (OS Signal)

A notification sent to a process by the OS or another process (e.g., SIGINT when the user presses Ctrl+C). Rust programs can handle signals via crates like `signal-hook`.

Thread

A unit of CPU execution within a process. Multiple threads share the same memory. Rust's type system (via `Send/Sync`) prevents the most common thread-safety bugs at compile time.

Virtual Table (vtable)

A table of function pointers used for dynamic dispatch with trait objects (`dyn Trait`). When you call a method on `Box<dyn Game>`, the runtime looks up the method in the vtable. Slight overhead compared to static dispatch but enables runtime polymorphism.

Game & Domain Concepts

Argon2 (in gaming context)

Used in Esoteric Wisdom (web project), not the RPG. Named after the noble gas argon. See *Argon2* in the crates section.

Boss

A powerful, named enemy that appears at the end of a dungeon or sub-area and provides a significant challenge. Iron Age RPG has bosses including `treant_lord`, `swamp_witch_queen`, and `crystal_elemental`.

Combat Engine

The subsystem that resolves fight encounters — calculating hit/miss, damage, status effects, and victory/defeat conditions. Lives in the `iron-age-combat` crate.

Crafting

The in-game mechanic of combining resources and components into new items. Defined in the `iron-age-crafting` crate with a recipe system.

Experience (XP / EXP)

Points awarded for completing quests and defeating enemies. Accumulating enough experience causes the player's character to level up, increasing stats.

Five-Card Draw

A poker variant where each player is dealt five cards face-down and may exchange some for new cards. Implemented in the Ultra Game Suite's Poker game.

Hand Evaluation

The process of ranking a poker hand. The Ultra Game Suite evaluates all standard hands: high card, pair, two pair, three of a kind, straight, flush, full house, four of a kind, straight flush, royal flush.

Immediate-Mode GUI

A GUI paradigm where the UI is fully re-described every frame from code, rather than maintaining a tree of persistent widget objects. `egui` is an immediate-mode GUI library. Simpler to reason about state changes; well-suited to game and tool UIs.

Inventory

The collection of items a player character carries. Managed by the `iron-age-inventory` crate, including capacity constraints and item stacking.

Iron Age

The historical period (approximately 1200–550 BC) characterised by the widespread use of iron tools and weapons. The thematic setting of the Iron Age RPG.

Levelling

The progression system where a character's experience points cross a threshold, triggering a level-up that improves their stats. Implemented in `iron-age-character`.

Minimax

An algorithm for two-player zero-sum games that finds the optimal move by simulating all possible future game states and assuming both players play perfectly. The Tic-Tac-Toe AI in the Ultra Game Suite uses minimax — it literally cannot be beaten, only drawn.

Parapsychology

The study of claimed psychic and paranormal phenomena such as telepathy, clairvoyance, and psychokinesis. A research topic on the Anomalous Inquiry platform.

Quest

A task or mission given to the player character in an RPG, with objectives, rewards, and narrative context. Iron Age RPG has 36 side quests across five sub-areas, managed by the `iron-age-narrative` crate.

Remote Viewing

A claimed practice of perceiving information about a distant target through extrasensory means. A documented research topic on the Anomalous Inquiry platform.

RPG Stats

Numerical attributes that define a character's capabilities: STR (Strength), INT (Intelligence), WIS (Wisdom), CON (Constitution), DEX (Dexterity), CHA (Charisma). Defined in the `Stats` struct of `iron-age-core`.

Side Quest

A non-essential quest that is optional relative to the main story arc but offers rewards, lore, and gameplay variety. Iron Age RPG has 36 side quests (5+ per area).

State Machine

A computational model consisting of a finite set of states and transitions between them based on inputs. Used throughout the Ultra Game Suite with Rust enums: e.g., `GameState::Menu`, `GameState::Playing`, `GameState::GameOver`. Rust's exhaustive match makes state machines safe.

Tarot

A set of 78 cards used in cartomancy (divination) and meditation. Each card has symbolic imagery and esoteric interpretations. Esoteric Wisdom features a tarot card reader with 15 historic decks.

Treant

A mythological tree creature (popularised in Dungeons & Dragons) — a large, ancient tree that is alive and sentient. `treant_lord` is the boss of Iron Age RPG's `ashwood_ancient_grove` sub-area.

Turn-Based Combat

A battle system where each participant takes turns acting sequentially, rather than acting simultaneously in real time. The Iron Age RPG combat engine is turn-based.

World Map

The spatial representation of the game world, including areas, sub-areas, connections, and traversal rules. Defined in `iron-age-world`. Currently includes named areas with associated enemies, quests, and bosses.

This glossary is a living document — terms should be added as the codebase and curriculum expand.